

Weekly Report: [W10]

General Information

- **Group ID:** 52
- **Group Name:** Group 52
- **Class:** 25A01
- **Project Name:** Super Mario Game
- **Date range:** 2026-08-09 – 2026-08-15
- **Github Repository:** <https://github.com/ndmhuy/SuperMarioGame.git>

Tasks Completed This Week

25125083 - Nguyễn Đình Minh Huy (Member A - Engine, Physics & Player/Item Entities)

Branch: `dev`

- **Task 1: Advanced Procedural Level Generation (`MapGenerator`):** Refined `MapGenerator` (`MapGenerator.cpp`) with multi-tier rolling elevation profiles, theme-specific overhead ceilings for Underground and Castle biomes, dynamic pit hazards with lava tiles, moving and falling platform spawning across large chasms with mathematical jump reachability guardrails, packaged item blocks (mushrooms, fire flowers, feathers encased in question blocks), 3 Star Coins distributed across graded elevation tiers, and an enemy threat-level pacing curve. Added a live ImGui tuning panel in `PlayingState` for real-time terrain re-generation. Verified with `verify_map_generator.cpp` (100% test pass).
- **Task 2: 3-World Campaign & Sub-Level Warp Pipe System:** Implemented the complete 3-world campaign architecture spanning World 1-1 (Grassland + Underground Sub-Vault), World 1-2 (Ice Cavern + Sky Canopy Sub-Vault), World 1-3 (Castle Fortress + Secret Lava Vault + Bowser Boss Encounter), and Bonus Stage 1. Developed `generate_game_levels.cpp` to procedurally synthesize all 7 level JSON maps with entrance/exit warp pipe linkage. Extended `SerializationUtils` and `LevelLoader` to serialize/deserialize all entity types, moving platforms, star coins, and warp pipe metadata (`pipeId` , `isEntrance` , `targetLevel` , `exitX` , `exitY`). Added seamless sub-level transitioning

in `PlayingState` via `loadLevelByPath()`, preserving player lives, coins, score, and power-up state.

- **Task 3: ImGui Campaign Navigation & Active Tube Warp Selector:** Updated level selection dropdowns across both "Gameplay Controls & Navigation" and "Physics & Input Test Playground" ImGui panels in `PlayingState` to provide instant navigation across all 7 campaign worlds and sub-levels. Implemented an interactive "Active Level Tube / Warp Pipe Destinations" dropdown selector and "Enter Tube" trigger button to allow live testing and verification of all warp pipe destinations.
 - **Task 4: Static Block & Item Physics Normalization:** Fixed static block gravity acceleration bug by overriding `getGravityMultiplier() const override { return 0.0f; }` in `Block.hpp` and updating `PhysicsEngine.cpp` to bypass gravitational acceleration for zero-gravity entities. Added `m_onGround` resting state to `Item.hpp` and implemented `resolveItemVsBlock` in `CollisionResolver` to ensure power-up mushrooms and items rest securely on platforms and question blocks without falling into the void.
 - **Task 5: Player Hitbox Gap Optimization & Combat Resolution:** Optimized player collision bounding box dimensions in `IPlayerState.cpp` from 32px to 24px width (\$24\times 30\$ Small, \$24\times 60\$ Super/Fire/Cape, \$14\times 14\$ Mini) with centered 4px side margins, enabling fluid traversal through 1-block (32px) vertical shafts and gap jumps without corner snagging. Refined `resolvePlayerVsEnemy` in `CollisionResolver.cpp` to accurately distinguish top head stomps (upward bounce impulse, stomp SFX, combo counter) from side touches (damage power-down, knockback, 2.0s invulnerability frames).
 - **Task 6: LevelLoader Robust File Path Resolution & Stream Reset:** Fixed `std::ifstream` failbit cascade bug in `LevelLoader.cpp` by explicitly calling `file.clear()` before each path candidate attempt. Expanded candidate fallback search paths (`build/assets/levels/`, `../build/assets/levels/`, `../assets/levels/`, `assets/levels/`) to guarantee deterministic level loading across diverse working directories. Synchronized all generated level JSON files across `assets/levels/` and `build/assets/levels/`.
 - **Task 7: Core Engine Stability & Shutdown Hardening:** Resolved static SFContext OpenGL destruction crashes (`SIGABRT`) on application exit by explicitly invoking `m_window.close()` in `Game.cpp` `shutdown()`. Hardened state destruction in `GameStateManager.cpp` with deferred state transition processing to eliminate mutex deadlocks during state transitions.
-

25125084 - Trần Gia Huy (Member B - Input, Sound & Gameplay Systems)

Branch: `B/feature/graphic-visual` → merged to `dev`

- **Task 1: Complete Entity Sprite Wiring & Aspect Ratio Scaling:** Wired sprite sheet frame animations across all entities (Player, Enemies, Items, Blocks, Hazards). Implemented uniform aspect ratio scaling for HUD coin counters and item entities to prevent sprite distortion. Configured 8-frame duration for POW block shockwaves, bottom-anchored bounce scaling for trampolines, and rendered squished pressed states for `PSwitch` even after collection/deactivation. Commits: `069817b`, `0bc21c3`, `2d5058e`, `83660b5`, `065557d`, `61e30bc`.
- **Task 2: Interactive Web Metadata Editor Studio:** Created `metadata_editor.html` and `edit_sprite_metadata.py` for managing sprite sheet JSON metadata. Implemented cursor-anchored zooming (`zoomAtPoint`), floating Position Finder HUD overlay displaying real-time pixel coordinates and 16x16 tile grid positions, interactive drag-to-draw frame bounding boxes, frame renaming, and step resizing. Validated 100% schema compatibility with TexturePacker JSON formats. Commits: `a0f1a3b`, `f43b2d0`.
- **Task 3: HUD Sprite Sheet & Star Coin Tracking Integration:** Integrated sprite assets into `Hud.cpp` and `Hud.hpp`: animated coin counter sprite (`coin_0..3`), collected vs missing Star Coin indicators (`big_coin` vs `big_coin_outline`), and dynamic character lives icon displaying active character's small idle sprite (Mario, Luigi, Toad, Peach). Synchronized star coin collection events and persistence in `PlayingState.cpp`.
- **Task 4: Directional Pipe Rendering Engine:** Implemented `PipeRenderer` (`PipeRenderer.cpp`) supporting 2-block wide AABB collision geometries, multi-part directional pipe head rendering (`pipe_green_head_left`, `pipe_red_head_left`, vertical top/bottom heads), and body extension tiling.

Branch: `B/feature/sfx-music`

- **Task 5: Game-Wide SFX & Background Music Integration:** Wired comprehensive sound effects and background music streaming into `SoundManager.cpp` and game systems. Subscribed audio playback to core game events: jump, stomp, coin pickup, block break, power-up/power-down, trampoline boing, stage clear, and game over. Added synthesized audio buffer fallback (`m_fallbackBuffer`) to safeguard against missing audio assets. Commits: `1002595`, `c383c19`, `9976209`.

Branch: `B/feature-fix/enemies-behavior`

- **Task 6: Advanced Enemy AI State Machines & Tuning:** Tuned behavior physics and state machines across all 11 enemy types in `Enemy.cpp`, `KoopaTroopa.cpp`, `KoopaParatroopa.cpp`, `Thwomp.cpp`, `Boo.cpp`, and `HammerThrowStrategy.cpp`. Implemented Thwomp teeth-gritting vibration (0.2s), rapid 700px/s slam, floor rest (0.5s), and slow -80px/s ascent; Boo gaze-detection (cower when faced vs 100px/s pursuit when

Mario turns away); HammerBro real-time player orientation and parabolic rotating hammer projectile throws (`hammer_black_0`) every 1.5s; and Koopa Troopa shell retraction, stomping, kicking, and high-speed sliding physics. Commit: `d954a66` .

- **Task 7: Enemy Visual Testing & Behavior Verification Suites:** Built dedicated interactive verification tool `verify_enemies_behavior.cpp` and enhanced `verify_enemies_visual.cpp` featuring 11 isolated room testing chambers with toggleable visual AABB bounding box overlays, zero-clipping ground clamping, and room-tailored ImGui teleportation presets. Commit: `d954a66` .
-

AI Usage Declaration

- Used AI to design procedural elevation profiles, cavern ceiling generation, and jump reachability guardrails in `MapGenerator` to guarantee 100% solvable level geometry.
 - Used AI to structure the 3-world campaign synthesis tool (`generate_game_levels.cpp`) and bidirectional warp pipe serialization in `SerializationUtils` and `LevelLoader` .
 - Used AI to diagnose and resolve the `std::ifstream` failbit multi-path probing bug in `LevelLoader` and static `SFContext` shutdown SIGABRT crash in `Game::shutdown()` .
 - Used AI to implement cursor-anchored zooming math and real-time tile grid calculation in the Web Metadata Editor Studio (`metadata_editor.html`).
 - Used AI to design the state machines and kinematics for Thwomp slam-rise cycles, Boo line-of-sight tracking, HammerBro parabolic projectile physics, and Koopa sliding shell momentum.
-

Tasks Planned for Next Week

- **Nguyễn Đình Minh Huy:** Implement Phase 11 features: Two-Player Mode (Luigi co-op split/shared camera tracking, independent input bindings) and Shadow Mario AI ghost recorder using circular packet ring buffer for delayed playback. Merge latest Member B branches (`B/feature/sfx-music` , `B/feature-fix/enemies-behavior`) into `dev` and conduct integrated playtesting. Write W11 report.
 - **Trần Gia Huy:** Complete In-Game Debug Console overlay (Command pattern execution for godmode, teleportation, item spawning, and speed tuning). Finalize Boss fight mechanics (Bowser fire breath and hammer barrage in World 1-3). Wire particle burst effects to entity combat events.
-

Issues & Resolutions

Issue 1: QuestionBlock and Static Blocks Accelerating into Void

- **Problem:** `QuestionBlock`, `BrickBlock`, and other block entities inherited from `Block` (which derives from `Entity`). Because `Block` did not override `getGravityMultiplier()`, it defaulted to `1.0f`. During runtime physics updates, `PhysicsEngine::applyGravity()` applied downward acceleration ($g = 1800 \text{ px/s}^2$) to static blocks, causing question blocks to detach from the level layout and accelerate into the void.
- **Resolution:** Overrode `getGravityMultiplier() const override { return 0.0f; }` in `Block.hpp`. Updated `PhysicsEngine::update()` to check `if (entity->getGravityMultiplier() > 0.0f)` before accumulating downward gravitational velocity, ensuring static blocks remain permanently anchored.

Issue 2: Spawned Items Slipping Through Solid Block Platforms

- **Problem:** Power-up `Mushroom` and `FireFlower` items dispensed from question blocks lacked a dedicated resting state flag. `CollisionResolver::resolveEntityVsTile()` only corrected terrain tile penetrations without zeroing accumulated downward velocity, allowing moving items to slip through block seams and plunge into the abyss.
- **Resolution:** Added `m_onGround` boolean flag to `Item.hpp`. Implemented `CollisionResolver::resolveItemVsBlock()` to handle item-to-block collision responses, setting vertical velocity $v_y = 0$ and `m_onGround = true`. Updated `PhysicsEngine::applyGravity()` to bypass gravity accumulation when an item is confirmed resting on a solid surface.

Issue 3: Player Hitbox Width Preventing 1-Block Chute Descent & Gap Traversal

- **Problem:** The player collision bounding box was initialized to 32×32 px (and 32×64 px for Super/Fire forms). Because grid tiles and vertical shaft openings are exactly 32 px wide ($1 \text{ tile} = 32 \text{ px}$), entering a 1-tile drop-down chute or jumping across 1-tile gaps caused micro-pixel corner snags against adjacent block AABBs, preventing Mario from entering pipes or falling through 1-block shafts.
- **Resolution:** Optimized player collision box dimensions in `IPlayerState.cpp` to 24×30 px for Small Mario, 24×60 px for Super/Fire/Cape Mario, and 14×14 px for Mini Mario, centered horizontally within the 32px sprite width with 4px margins on each side. This provides clean clearance for vertical descent and gap jumps without tile edge snagging.

Issue 4: LevelLoader `std::ifstream` Failbit Cascade on Multi-Path Probing

- **Problem:** In `LevelLoader::loadLevel()`, candidate paths were checked sequentially in a loop using `file.open(candidate)`. When the first candidate path failed to open, `std::ifstream` set its internal `failbit`. Subsequent calls to `file.open()` with valid fallback paths were silently ignored because the stream error state was not cleared, causing runtime failures when opening sub-level maps (e.g. `assets/levels/level_1_sub.json`).
- **Resolution:** Added `file.clear()` before each `file.open()` iteration in `LevelLoader.cpp`. Expanded fallback candidate search paths to include `build/assets/levels/`, `../build/assets/levels/`, `../assets/levels/`, and `assets/levels/`, ensuring level loading succeeds regardless of whether the executable is launched from the workspace root or the build directory.

Issue 5: Application Exit Crash from Static SFContext Destruction (`SIGABRT`)

- **Problem:** Closing the game window caused static destruction order collisions where SFML's internal OpenGL context (`SFContext`) was terminated while the active `sf::RenderWindow` still held OpenGL handles, producing an immediate `SIGABRT` crash during program exit.
- **Resolution:** Updated `Game::shutdown()` in `Game.cpp` to explicitly invoke `m_window.close()` before popping state stacks and shutting down manager singletons, ensuring all OpenGL context references are cleanly relinquished prior to static teardown.